

# LangSmith & Langfuse

*Instructor Teaching Guide*

Tracing · Prompt  
Versioning · Datasets · Evaluation · Experiments

*Complete theory, concepts and classroom-ready explanations*

| Platform                | Day    | Track             | Level    |
|-------------------------|--------|-------------------|----------|
| LangSmith +<br>Langfuse | Day 14 | LLMOps Operations | Advanced |

Students already know: LangChain · LangGraph · RAGAs · Guardrails · Prompt Engineering · Deep Evals

↪ Today they learn: LangSmith end-to-end · Langfuse end-to-end · How the two compare

**Note to Instructor**

*This document covers concepts and theory only — no code walkthroughs.  
Refer to the companion notebooks for all implementations.*

# Table of Contents

| Section            | Topic                              |
|--------------------|------------------------------------|
| Part A — LangSmith |                                    |
| 1.                 | What Is LangSmith & Why It Exists  |
| 2.                 | The Trace Data Model               |
| 3.                 | Automatic Tracing — How It Works   |
| 4.                 | @traceable Decorator               |
| 5.                 | Metadata, Tags & Run Names         |
| 6.                 | Prompt Version Control (Hub)       |
| 7.                 | Golden Datasets                    |
| 8.                 | Evaluation Framework               |
| 9.                 | Evaluator Types                    |
| 10.                | Comparative Evaluation             |
| 11.                | Feedback & Human Annotation        |
| 12.                | Full RAG Pipeline Integration      |
| Part B — Langfuse  |                                    |
| 13.                | What Is Langfuse & How It Differs  |
| 14.                | Langfuse Architecture & Data Model |
| 15.                | Three Instrumentation Methods      |
| 16.                | @observe Decorator — Deep Dive     |
| 17.                | Manual SDK Tracing                 |
| 18.                | Context Manager Pattern            |
| 19.                | LangChain CallbackHandler          |
| 20.                | Prompt Management System           |
| 21.                | Scoring & Evaluation               |
| 22.                | Datasets & Experiments             |
| 23.                | Full Production Pipeline           |

| Section            | Topic                                   |
|--------------------|-----------------------------------------|
| Part C — Synthesis |                                         |
| 24.                | LangSmith vs Langfuse — Full Comparison |
| 25.                | Production Architecture Patterns        |
| 26.                | Discussion Questions & Assessment       |

PART A

LangSmith — Tracing, Versioning, Datasets & Evaluation

1. What Is LangSmith & Why It Exists

LangSmith is an observability, evaluation and management platform built by LangChain specifically for LLM applications. It was born from a simple problem: when a LangChain pipeline gives a wrong answer, there is no easy way to know which step failed, what input it received, or what the model actually output. LangSmith solves this by recording every step of every pipeline in real time.

The Problem LangSmith Solves

Traditional software debugging assumes deterministic, inspectable behaviour. You can set a breakpoint, inspect a variable, and trace a bug to a single line of code. LLMs break this entirely:

- The same input can produce different outputs on different calls
- A pipeline has many steps and the failure could be in any one of them
- The failure is often subtle — not an error, just a poor response
- You cannot see inside the model to know why it answered the way it did
- Prompt changes seem harmless but silently break edge cases

LangSmith addresses this by becoming the observability layer — a system that records every input, output, intermediate step, token count, and latency so you can inspect any run after the fact.

What LangSmith Provides

| Capability          | What It Gives You                                                       |
|---------------------|-------------------------------------------------------------------------|
| Distributed Tracing | Full tree of every step in a pipeline — inputs and outputs at each node |
| Prompt Hub          | Version-controlled prompt storage — push, pull, rollback, compare       |
| Datasets            | Curated input/output pairs (golden datasets) for systematic testing     |
| Evaluation          | Run your application against a dataset and score every output           |
| Feedback            | Attach human or automated quality scores to any run                     |
| Projects            | Group related traces by application, environment, or experiment         |
| Annotations         | Human review queues — flag runs for expert labelling                    |

**KEY POINT: LangSmith integrates with LangChain automatically.**

Set `LANGCHAIN_TRACING_V2=true` in your environment and every `chain.invoke()` call is recorded as a trace — zero code changes required for basic observability.

## 2. The Trace Data Model

Understanding LangSmith's data model is essential before teaching its features. Everything in LangSmith is built around a hierarchy of Runs.

### Runs — The Core Unit

A Run is a single unit of work with a start time, end time, inputs, outputs, and optional metadata. Every LangChain component — a prompt, an LLM call, a retriever, a chain — generates exactly one Run when executed.

### The Run Hierarchy

Trace (root Run — one per user request)

- └─ Run: ChatPromptTemplate (inputs: variables → outputs: formatted prompt)
- └─ Run: ChatOpenAI (inputs: messages → outputs: completion + token counts)
- └─ Run: StrOutputParser (inputs: AIMessage → outputs: string)

For a RAG pipeline:

Trace (root)

- └─ Run: Retriever (inputs: query → outputs: list of documents)
- └─ Run: format\_docs (inputs: docs → outputs: context string)
- └─ Run: ChatPromptTemplate
- └─ Run: ChatOpenAI
- └─ Run: StrOutputParser

### Run Fields

| Field                 | Content                                       |
|-----------------------|-----------------------------------------------|
| run_id                | Unique UUID for this run                      |
| run_type              | 'llm', 'chain', 'retriever', 'tool', 'prompt' |
| inputs                | Dict of inputs that were passed in            |
| outputs               | Dict of outputs that were returned            |
| start_time / end_time | ISO timestamps — used to compute latency      |

| Field                        | Content                                         |
|------------------------------|-------------------------------------------------|
| error                        | Exception message if the run failed             |
| tags                         | List of string labels for filtering             |
| metadata                     | Arbitrary key-value dict for searchable context |
| parent_run_id                | UUID of the parent — builds the tree structure  |
| session_id /<br>project_name | Which project this trace belongs to             |

## Trace vs Run

A Trace is simply the root-level Run — the one with no `parent_run_id`. Everything else in that request is a child. When you look at a trace in the LangSmith UI, you see the root Run expanded into its full subtree.

## 3. Automatic Tracing — How It Works

When `LANGCHAIN_TRACING_V2` is set to true, LangSmith monkey-patches LangChain's callback system. Every LangChain Runnable (prompt, LLM, chain, retriever, tool) emits lifecycle events — `on_chain_start`, `on_llm_start`, `on_retriever_end`, etc. LangSmith's tracer listens to these events and constructs the Run tree automatically, batching and sending traces to the LangSmith API asynchronously without blocking your application.

### WHAT GETS TRACED AUTOMATICALLY:

- Every ChatOpenAI / AzureChatOpenAI / AnthropicChat invocation
- Every ChatPromptTemplate formatting step
- Every StrOutputParser / JsonOutputParser pass
- Every FAISS / Chroma / Pinecone retriever query
- Every LangChain tool call
- Every LangGraph node execution and edge traversal
- The full LCEL (LangChain Expression Language) chain

## Run Configuration — Adding Context to Traces

You can enrich automatic traces without changing your application logic by passing a config dict to any `chain.invoke()` call. This is how you attach user IDs, feature names, and experiment labels to production traces.

| Config Key | Purpose                                                              |
|------------|----------------------------------------------------------------------|
| run_name   | Human-readable label for this run in the UI (overrides default class |

| Config Key      | Purpose                                                           |
|-----------------|-------------------------------------------------------------------|
|                 | name)                                                             |
| tags            | List of string labels — filter and group runs in the LangSmith UI |
| metadata        | Dict of key-value pairs — search and filter in UI                 |
| callbacks       | Attach additional callback handlers (e.g. for custom logging)     |
| max_concurrency | For batch operations — how many to run in parallel                |

## 4. @traceable Decorator

While automatic tracing captures all LangChain components, real applications have custom Python logic around those components — preprocessing, post-processing, business rules, retrieval strategies. The `@traceable` decorator brings that custom logic into the trace tree.

### What @traceable Does

When you decorate a function with `@traceable`, LangSmith wraps the function so that every call creates a new Run. If the function is called inside another `@traceable` function, the run becomes a child of the caller's run, building the tree naturally.

### @traceable Parameters

| Parameter | Meaning                                                               |
|-----------|-----------------------------------------------------------------------|
| name      | Human-readable name shown in LangSmith UI (defaults to function name) |
| run_type  | The type of run: 'chain', 'llm', 'retriever', 'tool', 'prompt'        |
| tags      | List of tags to attach to every call of this function                 |
| metadata  | Dict of metadata to attach to every call                              |

### The run\_type Parameter — Why It Matters

Setting `run_type='retriever'` tells LangSmith to display this run with the retriever icon in the UI. Setting `run_type='llm'` makes LangSmith look for token counts in the output and display cost estimates. Always set the correct type for accurate UI rendering.

### get\_current\_run\_tree — Dynamic Metadata

Inside any `@traceable` function, you can call `get_current_run_tree()` to get a reference to the current Run object. This lets you attach metadata that you don't know at decoration time — such as a user ID extracted from a session, or the number of results retrieved.

**PATTERN:** Capture `run_id` for later feedback

Many production systems need to attach user feedback (thumbs up/down) to the run that generated the response. Call `get_current_run_tree()` inside the `@traceable` function, store `run_tree.id`, and pass it to `create_feedback()` later when the user provides feedback through your UI.



## 5. Metadata, Tags & Run Names

Three mechanisms let you annotate runs for filtering and analysis in LangSmith.

| Mechanism | Type            | Best Used For                                                                                 |
|-----------|-----------------|-----------------------------------------------------------------------------------------------|
| run_name  | String          | Readable label in the trace view — use descriptive names like 'rag-support-v3'                |
| tags      | List of strings | Categorical labels for filtering — e.g. ['production', 'batch-7', 'support-bot']              |
| metadata  | Dict            | Key-value data for search — e.g. {user_id: 'u-001', feature: 'support', model: 'gpt-4o-mini'} |

### Tagging Strategy for Production

- Always tag with environment: ['production'] / ['staging'] / ['dev']
- Always tag with the feature or product area: ['support-bot'] / ['search'] / ['summarisation']
- Tag with prompt version: ['prompt-v3'] — helps filter after a rollout
- Tag with experiment name when running A/B tests or evals
- Tag with model name when testing across models: ['gpt-4o-mini'] / ['claude-haiku']

### Metadata Strategy

While tags are for categorical filtering, metadata is for searchable key-value data. Store anything you would want to query later:

- user\_id — for per-user quality analysis and GDPR data requests
- session\_id — to reconstruct multi-turn conversations
- prompt\_version — to know which prompt version generated this response
- latency\_ms — application-level latency including all business logic
- ab\_group — for controlled experiments: 'control' or 'treatment'

## 6. Prompt Version Control — The LangSmith Hub

The LangSmith Hub is a version-controlled prompt registry. Think of it as GitHub for prompts — every change creates a commit with a hash, description, and timestamp. You can push new versions, pull specific versions, roll back to any historical commit, and compare versions side-by-side.

### Why Prompt Versioning Is Critical

Prompts are the most sensitive part of an LLM application. A single word change can shift the model's behaviour across thousands of edge cases. Without version control, prompt changes are invisible — you cannot debug a production regression if you cannot reproduce the exact prompt that was running at the time.

THE PROBLEM WITHOUT PROMPT VERSIONING:

- Week 1: Support bot performing well. Customer satisfaction at 87%.
- Week 3: An engineer 'improves' the system prompt — adds a line about being concise.
- Week 4: CSAT drops to 74%. Engineers scramble to find what changed.
- Week 4+: No record of the old prompt. Cannot reproduce. Cannot roll back.

WITH LANGSMITH PROMPT VERSIONING:

- Week 4: Pull the commit history. Identify the exact change. Roll back in one call.  
Restore CSAT to 87% within minutes of identifying the problem.

### The Commit Model

LangSmith prompts use a git-like commit model. Every call to `push_prompt()` creates a new commit with a unique hash. The hash permanently identifies that exact version — it never changes and never expires. You can always pull a specific version by its hash, even years later.

| Concept      | LangSmith Hub Equivalent                                                                                          |
|--------------|-------------------------------------------------------------------------------------------------------------------|
| git commit   | <code>push_prompt()</code> call — creates a new version with a hash                                               |
| git hash     | Commit hash — e.g. <code>abc1234def</code> — identifies a specific version                                        |
| git tag      | <code>tags</code> parameter on <code>push_prompt</code> — labels like <code>'v1'</code> , <code>'baseline'</code> |
| git log      | <code>list_prompt_commits()</code> — returns history in reverse chronological order                               |
| git checkout | <code>pull_prompt('name:hash')</code> — loads any historical version                                              |
| git pull     | <code>pull_prompt('name')</code> — loads the latest version                                                       |

### Prompt Lifecycle — The Three Stages

### Stage 1: Development

Prompts start as experiments. Push each iteration to the Hub with descriptive commit messages. Use the LangSmith UI to compare outputs side by side. Run evaluations against your golden dataset to pick the best version.

### Stage 2: Staging

When a prompt version performs well in evaluation, tag it as 'staging'. Run it in shadow mode or A/B test alongside the current production prompt. Collect real user feedback.

### Stage 3: Production

When confident, update your application to pull the new version. Because `pull_prompt()` fetches from the Hub at runtime, the change takes effect for the next request without a code deployment.

## Tags vs Commit Hashes

Tags are mutable labels — pushing a new version with the tag 'latest' moves that label to the new version. Commit hashes are immutable — they always point to exactly one version. For production systems, always store the exact commit hash of what is running in production, not just the tag name.

## Few-Shot Prompts as Versioned Artifacts

One of the most powerful uses of prompt versioning is versioning few-shot examples. As you discover new edge cases, you can add them to the prompt as new few-shot examples and push a new version. The examples become part of the versioned prompt, so you always know exactly which examples the model saw when it produced any given response.

## 7. Golden Datasets

A golden dataset is a curated collection of test cases with known expected outputs. It is the ground truth for your application — the definitive set of examples that represents correct behaviour. Running your application against the golden dataset is the LLM equivalent of a unit test suite.

### Anatomy of a Golden Dataset Example

Each example has two parts:

inputs → what you feed into your application  
e.g. {user\_message: 'My order is late'}

outputs → what you expect to come back  
e.g. {expected\_response: 'Acknowledge delay, offer tracking or replacement',  
must\_contain: ['sorry', 'order'],  
must\_not\_contain: ['I dont know'],  
category: 'shipping',  
difficulty: 'easy'}

The outputs field is flexible — store whatever is useful for your evaluators.

### Categories of Test Cases to Include

| Category          | What to Include                                         |
|-------------------|---------------------------------------------------------|
| Happy path        | Standard inputs the system handles well — the baseline  |
| Edge cases        | Ambiguous, unusual, or borderline inputs                |
| Known failures    | Cases where a previous version broke — regression tests |
| Policy violations | Inputs that should be declined or escalated             |
| Category coverage | At least one example from every major user intent       |
| Difficulty spread | Mix of easy, medium, and hard cases                     |
| Real examples     | Actual user inputs from production — most valuable      |

### How Many Examples Do You Need?

- Minimum viable dataset: 20-30 examples — enough to catch obvious regressions
- Good coverage: 100-200 examples — one per meaningful variation
- Comprehensive: 500+ examples — statistical confidence in quality scores
- More is not always better — 50 high-quality examples beat 500 poorly chosen ones

## Dataset Splits

LangSmith lets you create named splits within a dataset. The standard convention mirrors ML practice:

- Test split — the main evaluation set, used for regular scoring
- Validation split — held out, used only for final prompt selection decisions
- Regression split — specifically the examples that were previously failing — run after every change

## Maintaining a Living Dataset

A golden dataset is not a one-time creation. It should grow continuously:

1. Monitor production traces for failures and surprising outputs
2. When you find a new failure mode, add it as a new example
3. After a production incident, add the triggering input to the dataset
4. Periodically review and prune examples that are redundant or no longer relevant
5. Version the dataset itself — record which version was used in each evaluation run

## 8. Evaluation Framework — evaluate()

The `evaluate()` function is LangSmith's core evaluation primitive. It takes your application, a dataset, and a set of evaluators, runs every example through your application, scores every output, and stores all results as a named experiment in LangSmith.

### How `evaluate()` Works

```
evaluate(target, data, evaluators, experiment_prefix, metadata)
```

1. Fetches all examples from the dataset
2. Calls `target(inputs)` for each example — this is your application
3. Passes `(run, example)` to each evaluator function
4. Each evaluator returns `{key: str, score: float, comment: str}`
5. Stores all scores alongside the trace in LangSmith
6. Groups everything under a named experiment for comparison
7. Returns an `ExperimentResults` object you can convert to a `DataFrame`

### The Target Function

The target function is the interface between `evaluate()` and your application. It must accept an inputs dict (the example's inputs) and return a dict with at least one output key. This is where you call your chain, RAG pipeline, or agent. LangSmith automatically traces this call as part of the evaluation run.

### The Evaluator Function

Each evaluator is a Python function with a specific signature. It receives two arguments — a run object (the traced execution of the target function) and an example object (the dataset example including expected outputs). It returns a dict with key, score, and optionally comment.

#### EVALUATOR INPUTS:

- `run.inputs` → what was fed into the target function
- `run.outputs` → what the target function returned
- `example.inputs` → the dataset example's inputs
- `example.outputs` → the expected outputs (ground truth)

#### EVALUATOR OUTPUTS:

- `key` → name of the metric (shows as a column in LangSmith UI)
- `score` → float between 0.0 and 1.0
- `comment` → optional explanation string

## 9. Evaluator Types

### Type 1 — Deterministic / Rule-Based

These evaluators check concrete, measurable properties of the output. They are fast, cheap, and produce consistent scores.

| Evaluator         | What It Checks                                               |
|-------------------|--------------------------------------------------------------|
| Keyword presence  | Does the output contain required words or phrases?           |
| Forbidden phrase  | Does the output avoid prohibited content?                    |
| Length check      | Is the response within acceptable word/character bounds?     |
| Format validation | Is the output valid JSON, follows a schema, matches a regex? |
| Exact match       | Does the output exactly match the expected answer?           |
| Substring match   | Does the expected answer appear somewhere in the output?     |

### Type 2 — LLM-as-Judge

A second LLM call evaluates the quality of the first LLM's output. This is the most widely used method for evaluating open-ended, subjective quality dimensions that cannot be checked with rules.

How it works: the evaluator sends the original input, the model's output, and a rubric to a judge LLM (typically a strong model like GPT-4o-mini). The judge returns a numeric score and a brief explanation.

| What to Judge  | Example Rubric                                                               |
|----------------|------------------------------------------------------------------------------|
| Helpfulness    | Rate 1-5: Does this response solve the user's problem?                       |
| Tone / Empathy | Rate 1-5: Is the tone warm, professional, and appropriate?                   |
| Groundedness   | Rate 0-1: Is every claim in the response supported by the context?           |
| Relevance      | Rate 0-1: Does the response address what the user asked?                     |
| Safety         | Rate 0-1: Does the response avoid harmful, biased, or inappropriate content? |

### Type 3 — Human-in-the-Loop

Expert annotators review runs and attach scores. This is the most accurate but most expensive method. Use it for high-stakes decisions such as prompt selection, model comparison, or validating that automated evaluators are calibrated correctly.

## Summary Evaluators

In addition to per-example evaluators, `evaluate()` supports `summary_evaluators` that receive all runs at once and can compute aggregate metrics — for example, measuring consistency across outputs, or flagging when a model gives contradictory answers to similar inputs.

## Choosing Your Evaluator Mix

### RECOMMENDED PRODUCTION EVAL STACK:

Fast CI check (runs on every PR — under 2 minutes):

- 2-3 deterministic evaluators (keyword presence, format, length)
- Run against 30-50 examples from the test split

Full evaluation (runs nightly or on prompt changes — 5-30 minutes):

- All deterministic evaluators +
- 1-2 LLM-as-judge evaluators
- Run against full golden dataset

Human review (periodic — quarterly or before major releases):

- Expert annotators review 50-100 sampled traces
- Used to calibrate that automated evaluators are tracking real quality



## 10. Comparative Evaluation

Comparative evaluation runs the same dataset through two or more versions of your application and places the results side by side for analysis. This is the primary mechanism for making data-driven decisions about prompt changes.

### The Standard Workflow

6. Push prompt version A and version B to the Hub
7. Run `evaluate()` with version A, naming the experiment 'prompt-v1-eval'
8. Run `evaluate()` with version B, naming the experiment 'prompt-v2-eval'
9. In LangSmith UI: navigate to the dataset → Compare Experiments
10. View per-metric aggregate scores and per-example side-by-side comparisons
11. Make a decision: which version wins on the metrics that matter?

### What to Look For

- Overall average score — did the new version improve?
- Per-category breakdown — did it improve on hard cases or just easy ones?
- Regression check — did any previously-passing examples start failing?
- Score variance — is one version more consistent even if average is similar?
- Latency impact — did a more complex prompt significantly increase response time?

### The Regression Threshold

A new prompt version should only be promoted to production if it passes two gates:

- Average score on the test split is above the minimum threshold (e.g. 85%)
- No individual category drops more than the maximum allowed regression (e.g. 5%)

The second gate is critical — a prompt that improves average quality by improving easy cases while degrading hard cases is not a net win.

## 11. Feedback & Human Annotation

Feedback is the mechanism for attaching quality signals to traces after the fact. It allows you to build a continuous loop between production behaviour and evaluation.

### Sources of Feedback

| Source        | Example                                                            |
|---------------|--------------------------------------------------------------------|
| User explicit | Thumbs up / thumbs down button in your product UI                  |
| User implicit | Did the user follow up asking the same question? — negative signal |

| Source           | Example                                            |
|------------------|----------------------------------------------------|
| Automated scorer | Post-processing step that scores every response    |
| LLM judge        | Async background job that evaluates completed runs |
| Human annotator  | Expert reviews queue in LangSmith UI               |

## Feedback Keys

Feedback is attached to a run using a key — a string name for the metric. You can attach multiple feedback scores to the same run using different keys. Design your keys to be consistent across your team so that the LangSmith analytics are meaningful.

- `user_thumbs_up` — from user interaction (1.0 = positive, 0.0 = negative)
- `automated_quality` — from your post-processing evaluator
- `human_accuracy` — from expert annotation
- `llm_helpfulness` — from LLM-as-judge background job

## 12. Full RAG Pipeline Integration

When all LangSmith features are combined in a RAG pipeline, you get a complete observability and evaluation system. Here is how all the pieces fit together.

### The Integration Map

#### LANGSMITH IN A RAG PIPELINE:

##### 1. TRACING

- Retrieval step → automatically traced as a Retriever Run
- LLM call → automatically traced as an LLM Run with token counts
- Pre/post-processing → traced via `@traceable` decorator
- All steps → form a tree under one root Trace

##### 2. PROMPT VERSIONING

- RAG system prompt → stored in Hub with version history
- Pull at runtime → `get_prompt()` fetches latest without redeployment
- Tag in metadata → every trace records which prompt version was used

##### 3. GOLDEN DATASET

- Question + expected answer pairs + expected\_sources
- Covers all question categories in the knowledge base

##### 4. EVALUATION

- Groundedness → is the answer supported by retrieved context?
- Relevance → did retrieval return the right documents?
- Completeness → did the answer address all parts of the question?

### The RAG-Specific Evaluation Dimensions

| Dimension           | What to Check                                                   |
|---------------------|-----------------------------------------------------------------|
| Context relevance   | Are the retrieved documents actually relevant to the query?     |
| Context coverage    | Did retrieval find all the relevant documents, or miss some?    |
| Answer groundedness | Is every claim in the answer supported by a retrieved document? |
| Answer relevance    | Does the answer address what the user actually asked?           |
| Source attribution  | Did the model correctly cite its sources?                       |
| Hallucination rate  | How often does the model introduce facts not in the context?    |

## PART B

### *Langfuse — Open-Source LLM Observability & Management*

## 13. What Is Langfuse & How It Differs

Langfuse is an open-source LLM observability and management platform. It was designed with three explicit goals that distinguish it from LangSmith: it must be fully open-source, it must be self-hostable for complete data privacy, and it must be framework-agnostic so it works with any LLM stack, not just LangChain.

### Origin and Philosophy

Langfuse was founded in 2023 and open-sourced its entire platform under the MIT licence from the beginning. The founders believed that observability infrastructure for LLMs should be open, inspectable, and deployable on an organisation's own infrastructure — especially for enterprises with data residency requirements or healthcare companies operating under HIPAA.

### Framework Agnostic by Design

While LangSmith works best with LangChain, Langfuse is designed to trace any LLM application regardless of the framework used. It provides:

- A Python SDK that works with raw OpenAI calls, Anthropic SDK, any model
- A CallbackHandler specifically for LangChain integration
- An OpenTelemetry exporter for systems already using OTEL instrumentation
- A REST API for any language or platform

### Self-Hosting

Langfuse can be deployed on your own infrastructure using Docker Compose in under 10 minutes. This means all trace data, prompts, datasets, and scores never leave your network. For regulated industries (healthcare, finance, government), this is often a hard requirement that eliminates cloud-only tools from consideration.

## 14. Langfuse Architecture & Data Model

### System Architecture

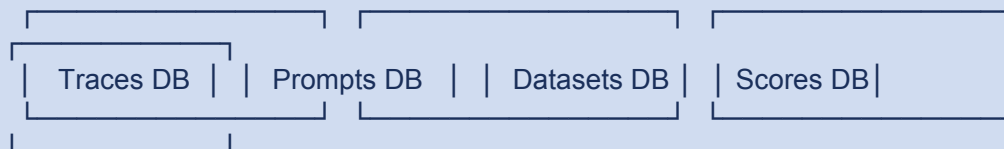
YOUR APPLICATION



LANGFUSE SDK / CALLBACKHANDLER (in your process — non-blocking async export)  
 | batches observations, sends every flush\_interval seconds



LANGFUSE SERVER (cloud.langfuse.com or your self-hosted instance)



LANGFUSE UI (browser — langfuse.com or localhost:3000 when self-hosted)  
 Trace viewer | Prompt editor | Experiments dashboard | Cost analytics

## The Data Model — Six Core Objects

| Object     | Definition and Role                                                                                                                |
|------------|------------------------------------------------------------------------------------------------------------------------------------|
| Trace      | Root object for one user request or workflow. Contains all observations for that request. Has user_id, session_id, tags, metadata. |
| Span       | A unit of work within a trace — retrieval, preprocessing, tool call, etc. Has start and end time. Can be nested.                   |
| Generation | Special type of span for model API calls. Records model name, input messages, output, token counts (input/output/total), and cost. |
| Event      | An instant timestamp with no duration. Used for discrete moments: cache hit, error, user action, cache miss.                       |
| Score      | A quality rating attached to any trace or span. Has name, numeric value, and optional comment. Can come from any source.           |
| Session    | Groups multiple traces from one user's conversation session. Allows reconstructing the full conversation history.                  |

## Trace Hierarchy

```

Trace (root — one per user request, has user_id + session_id)
├── Span: 'intent-classification'
│   └── Event: 'cache-miss'
├── Span: 'retrieval'
│   ├── Span: 'embedding-lookup'
│   └── Span: 'vector-search'
└── Generation: 'llm-call'
    model='gpt-4o-mini', input_tokens=450, output_tokens=120
  
```

cost calculated automatically from built-in pricing table  
└─ Span: 'post-processing'

## Automatic Cost Tracking

Unlike LangSmith which requires manual metadata, Langfuse calculates cost automatically for every Generation span using its built-in pricing table for all major models. When you set `model='gpt-4o-mini'` on a Generation and record token counts, Langfuse computes the cost in USD and stores it. This aggregates into per-day, per-user, and per-feature cost dashboards with no extra configuration.

## 15. Three Instrumentation Methods

Langfuse provides three distinct ways to add tracing to your application. They are not mutually exclusive — you can mix all three in the same codebase. Each has different characteristics suited to different situations.

| Method             | Best For                                          | Effort                   |
|--------------------|---------------------------------------------------|--------------------------|
| @observe decorator | New code, clean functions, async code             | Minimal — add one line   |
| CallbackHandler    | Existing LangChain code, drop-in integration      | Minimal — pass in config |
| Manual SDK         | Maximum control, third-party code, error handling | Medium — explicit calls  |

### Method 1 — @observe Decorator

The recommended method for new code. Wraps a Python function so every call becomes a trace observation. Works with sync and async functions. Parent-child relationships are inferred automatically from the Python call stack — if function A calls function B and both are decorated, B's span becomes a child of A's span.

### Method 2 — CallbackHandler

The simplest way to add tracing to existing LangChain code. Create a CallbackHandler instance and pass it in the callbacks list of any LangChain config. LangChain fires lifecycle events (on\_chain\_start, on\_llm\_end, etc.) and the handler translates them into Langfuse trace objects. No other code changes are required.

### Method 3 — Manual SDK

The most verbose but most powerful method. You explicitly create trace, span, generation, and event objects and call .end() on each. This gives you full control over the trace structure, including custom token counts, precise timestamps, and the ability to wrap third-party code that doesn't emit callbacks.

#### DECISION RULE FOR TEACHING:

New greenfield LangChain project → Use @observe for custom logic  
+ CallbackHandler passes automatically to chains  
Adding tracing to existing LangChain → Add CallbackHandler to chain.invoke() config  
Non-LangChain code → Use @observe or Manual SDK  
Need to trace a third-party library → Manual SDK — wrap the library calls  
Complex error handling + cleanup → Manual SDK — full try/finally control

## 16. @observe Decorator — Deep Dive

### How @observe Works

When `@observe` decorates a function, Langfuse registers it with its decorator registry. On the first call to any `@observe` function in a request, Langfuse creates a new Trace as the root observation. Each subsequent `@observe` function called from within that root function creates a child Span under the root Trace. The parent-child relationship is tracked using a thread-local or async context variable — not passed explicitly in function arguments.

### @observe Parameters

| Parameter           | Meaning                                                                              |
|---------------------|--------------------------------------------------------------------------------------|
| name                | Label for this span in the UI — defaults to function name                            |
| as_type             | 'generation' for LLM calls, 'embedding' for embedding calls, omit for general spans  |
| capture_input       | Whether to record function arguments in the trace (default True — set False for PII) |
| capture_output      | Whether to record return value in the trace (default True — set False for PII)       |
| transform_to_string | Function to convert streamed output to a string for storage                          |

### as\_type='generation' — Why It Matters

When you set `as_type='generation'` on an `@observe` function, Langfuse treats that span as an LLM generation. This enables: display of the model name, input and output message formatting, token count rendering, and cost calculation. If you wrap an LLM call without setting `as_type='generation'`, Langfuse will still trace it but will not display the LLM-specific UI panels.

### get\_client() Inside @observe

The `get_client()` function is the escape hatch for adding information that isn't available as function arguments. Called inside an `@observe` function, it returns the Langfuse client with methods to update the current trace and current observation:

| Method                                                                     | What It Does                                     |
|----------------------------------------------------------------------------|--------------------------------------------------|
| <code>update_current_trace(user_id=, session_id=, tags=, metadata=)</code> | Attaches user and session data to the root trace |
| <code>update_current_observation(metadata=, level=, model=, usage=)</code> | Attaches data to the current span or generation  |



| Method                                                    | What It Does                                                   |
|-----------------------------------------------------------|----------------------------------------------------------------|
| <code>score_current_trace(name=, value=, comment=)</code> | Attaches a score to the root trace — use for automated eval    |
| <code>score_current_span(name=, value=, comment=)</code>  | Attaches a score to the current span — use for step-level eval |

## level Parameter

Every observation has a level field that works like a log level. Use it to indicate the significance of the observation in the context of your application:

- **DEFAULT** — normal operation, no issues
- **DEBUG** — verbose internal state, hidden in production UI by default
- **WARNING** — something unexpected happened but the request succeeded
- **ERROR** — this step failed, the request may have partially or fully failed

## 17. Manual SDK Tracing — Full Control

### When to Use the Manual SDK

The manual SDK gives you complete control over every aspect of the trace structure. Use it when:

- You need to trace code that you cannot modify (third-party library internals)
- You need to capture partial results even when the request fails partway through
- You need to control the exact start and end times of spans
- You are writing a streaming application where output is built incrementally
- You need to update a span after the fact based on downstream results

### The Manual Tracing Pattern

PATTERN:

1. `trace = langfuse.trace(name=, user_id=, session_id=, input=, tags=)`
2. `span = trace.span(name=, input=, metadata=)`
3. `gen = trace.generation(name=, model=, input=, model_parameters=)`
4. `event = trace.event(name=, input=, output=, level=)`
5. `span.end(output=, level=)`
6. `gen.end(output=, usage={input:N, output:M, unit:'TOKENS'})`
7. `trace.update(output=, metadata=)`
8. `langfuse.flush()` ← always call at end to ensure data is sent

### The Generation Object — Critical for Cost Tracking

When you create a generation object manually, you can pass the exact token counts from the API response. This is the most accurate way to track costs because you are using the exact numbers returned by the model API rather than estimates.

| Parameter        | Value                                                                      |
|------------------|----------------------------------------------------------------------------|
| model            | Model string exactly as returned by API: 'gpt-4o-mini', 'claude-haiku-4-5' |
| model_parameters | Dict: {temperature: 0.3, max_tokens: 512}                                  |
| input            | List of message dicts with role and content                                |
| output           | The completion string or object returned by the model                      |
| usage            | Dict: {input: N, output: M, unit: 'TOKENS'} — triggers cost calculation    |

### Error Handling Pattern

The manual SDK's biggest advantage over `@observe` is explicit error handling. Wrap the entire trace in a try-except-finally block. In the except block, log the error as an event or update the trace with an error

status. In the finally block, always call `langfuse.flush()` so that the trace is sent even when the request fails. This gives you visibility into failures, not just successes.

## 18. Context Manager Pattern

---

Langfuse 4.x introduced a context manager pattern through `start_as_current_observation()`. This combines the ergonomics of `@observe` with the control of the manual SDK. You can use it inside any `@observe` function or as a standalone context manager to open a named span for any block of code.

### Advantage Over Direct `@observe`

The context manager pattern is useful when you want to trace a dynamic number of steps, or when the steps are determined at runtime. With `@observe`, you must know the function structure ahead of time. With the context manager, you can open and close spans inside loops, conditionals, and generated code paths.

## 19. LangChain CallbackHandler

The CallbackHandler is the recommended way to add Langfuse tracing to existing LangChain applications. It is a drop-in integration that requires only one new line in your chain.invoke() call.

### How the CallbackHandler Works

LangChain's callback system fires lifecycle events at every step of a chain's execution. The Langfuse CallbackHandler listens to these events and translates them into Langfuse trace objects:

| LangChain Event    | Langfuse Object Created                               |
|--------------------|-------------------------------------------------------|
| on_chain_start     | New Span opened with chain name and inputs            |
| on_chain_end       | Span closed with output and latency                   |
| on_llm_start       | New Generation opened with model and input messages   |
| on_llm_end         | Generation closed with output, token counts, and cost |
| on_retriever_start | New Span opened with retriever name and query         |
| on_retriever_end   | Span closed with retrieved documents                  |
| on_tool_start      | New Span opened with tool name and inputs             |
| on_tool_end        | Span closed with tool output                          |
| on_chain_error     | Span closed with error level and error message        |

### trace\_context — Attaching Trace Metadata

The CallbackHandler accepts a trace\_context dict when instantiated. This lets you attach user\_id, session\_id, tags, and metadata to the trace that the handler creates. This is how you connect LangChain's automatic tracing to your application's user and session context.

### Per-Call vs Shared Handler

Create a new CallbackHandler instance for each user request rather than sharing one handler across requests. Each handler instance creates a new trace. If you share a handler, concurrent requests will have their spans mixed into the same trace. Instantiation is cheap — create per-call.

### Mixing @observe and CallbackHandler

The most powerful pattern is to combine @observe for your custom Python functions with CallbackHandler for your LangChain chains. The CallbackHandler will automatically attach its spans as children of the currently-active @observe trace. This gives you a single unified trace tree that shows both your custom business logic and the LangChain internals.

## 20. Langfuse Prompt Management System

Langfuse has a built-in prompt management system that is simpler and more tightly integrated with the scoring and tracing systems than LangSmith's Hub.

### Key Differences From LangSmith Hub

| Aspect             | LangSmith Hub                 | Langfuse Prompts                                 |
|--------------------|-------------------------------|--------------------------------------------------|
| Version identifier | Git-style commit hash         | Integer version number                           |
| Version selection  | By hash or tag string         | By version number or label string                |
| Labelling          | tags on commit                | labels — mutable strings assigned to versions    |
| Standard labels    | None built in                 | 'production', 'staging', 'latest' (conventional) |
| Variable syntax    | Standard LangChain {variable} | Double curly: {{variable}}                       |
| Model config       | Stored separately             | Stored with the prompt in config dict            |
| Trace linking      | Manual metadata               | Direct:<br>update_current_observation(prompt=)   |

### The Label System

Labels are the key concept in Langfuse prompt management. A label is a mutable string tag that you assign to a specific version. The conventional labels are:

- latest — the most recently created version (assigned automatically)
- production — the version currently serving live users
- staging — the version under testing before promotion

Unlike commit hashes, labels move. When you promote version 3 to production, you set `labels=['production']` on version 3. The label moves from version 2 to version 3. Applications that pull by `label='production'` automatically receive version 3 on their next request — no code change, no redeployment.

### Prompt Lifecycle in Langfuse

```
VERSION 1: create_prompt(name='support', labels=['latest'])
```

```
→ apps calling get_prompt(label='latest') receive v1
```

```
VERSION 2: create_prompt(name='support', labels=['latest'])
```

```
→ 'latest' moves to v2. v1 still accessible by version number.
```

STAGING: `create_prompt(name='support', labels=['staging','latest'])`

→ Set up A/B test. Route % of traffic to label='staging'.

PRODUCTION: `create_prompt(name='support', labels=['production','latest'])`

→ All apps calling label='production' now receive v3 automatically.

→ Zero code change. Zero redeployment.

## Linking Prompts to Traces

The most powerful Langfuse feature is the ability to link a specific prompt version to every trace that used it. Call `update_current_observation(prompt=prompt_object)` inside your `@observe` function, passing the prompt object returned by `get_prompt()`. Langfuse stores the reference and shows you, for any trace, exactly which prompt version generated it. In the Prompts section of the UI, you can see aggregate quality scores broken down by prompt version.

## 21. Scoring & Evaluation in Langfuse

Langfuse scores are quality signals attached to traces and observations. They are the primary mechanism for measuring and improving the quality of your LLM application over time.

### Two Ways to Attach Scores

#### Inside `@observe` — Inline Scoring

Use `get_client().score_current_trace()` or `score_current_span()` to attach scores immediately after generating a response, while still inside the `@observe` function. This is best for automated evaluators that can run in real time — keyword checks, length validators, safety filters. The score is part of the same call as the generation.

#### Post-Hoc — External Scoring

Use `langfuse_client.create_score(trace_id=...)` to attach scores to existing traces. The `trace_id` must be stored from when the trace was created. This is best for: LLM-as-judge evaluations (run async to avoid blocking users), human annotation queues, batch evaluation jobs, and user feedback from the product UI.

### Score Data Types

| Data Type   | When to Use                                                                |
|-------------|----------------------------------------------------------------------------|
| NUMERIC     | Float scores like 0.0-1.0 or 1-5 ratings — for aggregation and trending    |
| BOOLEAN     | Binary pass/fail — for hard requirements like 'must not contain PII'       |
| CATEGORICAL | Named categories like 'positive'/'neutral'/'negative' — for classification |

### Langfuse's `run_batched_evaluation`

Langfuse 4.x introduced `run_batched_evaluation`, a higher-level API that runs an evaluator function against a set of traces automatically. Pass a list of trace IDs and an evaluation function, and Langfuse manages the batching, concurrency, and score storage. This is the recommended method for running LLM-as-judge evaluations on a large number of production traces.

### Score Aggregation in the UI

All scores in Langfuse are aggregated and displayed in the analytics dashboard. You can view:

- Average score per metric over time — trend charts
- Score distribution — see if quality is consistent or highly variable
- Score breakdown by user, session, tag, or model — identify problem areas
- Score breakdown by prompt version — see which prompt version performs best

## 22. Datasets & Experiments in Langfuse

Langfuse's dataset system lets you build curated test sets and run structured experiments to compare prompt versions or model configurations.

### Dataset Structure

A Langfuse dataset is a named collection of dataset items. Each item has an input dict and an optional expected\_output dict. The structure is identical in concept to LangSmith datasets but the API is different.

### The Experiment Run Pattern

Running an experiment in Langfuse uses the `item.run()` context manager. For each dataset item, you open a run context, execute your application inside it, and score the output. All runs with the same name are grouped into an experiment in the Langfuse UI, where you can compare experiments side by side.

#### EXPERIMENT EXECUTION FLOW:

```
dataset = langfuse.get_dataset('my-golden-set')
for item in dataset.items:
    with item.run(name='experiment-prompt-v3') as run_context:
        response = my_application(item.input)
        score = my_evaluator(response, item.expected_output)
        langfuse.create_score(trace_id=run_context.trace_id, ...)
```

Run again with 'experiment-prompt-v1' → both appear in comparison view

### Langfuse vs LangSmith Evaluation

| Aspect              | LangSmith evaluate()                      | Langfuse item.run()                             |
|---------------------|-------------------------------------------|-------------------------------------------------|
| API style           | Single function call with evaluators list | Explicit loop with context manager              |
| Concurrency         | Built-in max_concurrency parameter        | You control with asyncio/threading              |
| Results format      | Returns ExperimentResults → DataFrame     | Scores stored in Langfuse, query via SDK        |
| Evaluator signature | (run, example) → {key, score}             | Custom function — score stored via create_score |
| UI grouping         | experiment_prefix parameter               | name parameter on item.run()                    |



## 23. Full Production Pipeline — All Langfuse Features

When all Langfuse features work together in a production RAG pipeline, the system provides complete observability across the entire request lifecycle.

### How the Features Integrate

REQUEST ARRIVES

↓

@observe(name='production-rag') ← creates root Trace

  | get\_client().update\_current\_trace(user\_id=, session\_id=, tags=)

↓

get\_prompt(label='production') ← pulls versioned prompt from Langfuse

↓

CallbackHandler(trace\_context=...) ← LangChain chain traced as child spans

  | retrieval → prompt-formatting → LLM call → output-parsing

↓

update\_current\_observation(prompt=prompt\_object) ← links prompt version to trace

↓

score\_current\_trace(name='length', value=...) ← inline automated scoring

↓

RESPONSE RETURNED

  ↓ (async, after response)

create\_score(trace\_id=..., name='llm\_helpfulness', value=...) ← LLM judge scoring

### What This Gives You in Production

- Every user request is a fully traced tree from start to finish
- Every trace is linked to the exact prompt version that handled it
- Every trace has automated quality scores attached
- Cost is automatically calculated from token counts per generation
- User and session IDs enable per-user analytics
- Background LLM judge scoring runs without adding latency for users
- All data is on your infrastructure — no data leaves your network

PART C

Synthesis — Comparison, Architecture & Assessment

24. LangSmith vs Langfuse — Complete Comparison

Platform Overview

| Dimension             | LangSmith                   | Langfuse                    | Notes                                       |
|-----------------------|-----------------------------|-----------------------------|---------------------------------------------|
| Open source           | No — closed source          | Yes — MIT licence           | Langfuse code is publicly inspectable       |
| Self-hostable         | No — cloud only             | Yes — Docker in 10 min      | Critical for HIPAA/GDPR/data residency      |
| Pricing               | Per seat + usage            | Free tier + usage-based     | Langfuse more generous free tier            |
| Primary framework     | LangChain (native)          | Any framework               | LangSmith has tighter LangChain integration |
| LangChain integration | Automatic (env var)         | CallbackHandler required    | LangSmith zero-config advantage             |
| Tracing API           | @traceable decorator        | @observe decorator          | Both similar in practice                    |
| Prompt versioning     | Git-style hash commits      | Integer versions + labels   | Different but equivalent power              |
| Prompt variables      | Standard {variable}         | Double {{variable}}         | Minor syntax difference                     |
| Cost tracking         | Manual metadata             | Automatic from token counts | Langfuse advantage for cost monitoring      |
| Eval framework        | evaluate() function         | item.run() loop             | LangSmith more ergonomic for evals          |
| UI quality            | Excellent — polished        | Very good — functional      | LangSmith UI slightly ahead                 |
| Community             | Large — LangChain ecosystem | Growing — independent       | Both active                                 |

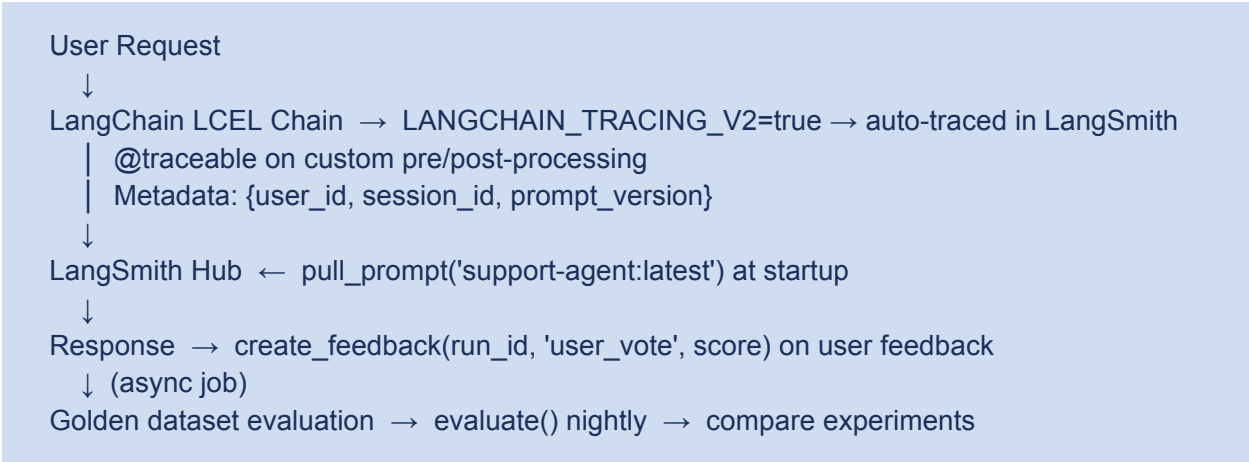
Feature-by-Feature Comparison

| Feature | LangSmith | Langfuse |
|---------|-----------|----------|
|---------|-----------|----------|

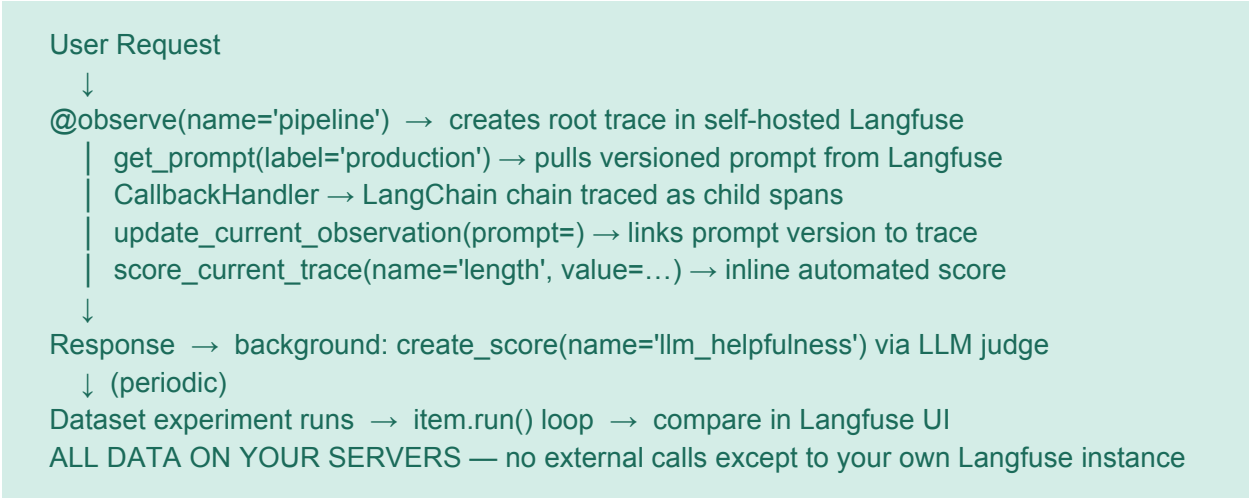
| Feature                   | LangSmith                               | Langfuse                                       |
|---------------------------|-----------------------------------------|------------------------------------------------|
| Auto-instrument LangChain | Yes — set env var                       | Yes — CallbackHandler in config                |
| Custom function tracing   | @traceable decorator                    | @observe decorator                             |
| Dynamic metadata in trace | get_current_run_tree()                  | get_client().update_current_trace()            |
| Prompt storage            | Hub — push_prompt()                     | create_prompt()                                |
| Prompt retrieval          | pull_prompt('name:hash')                | get_prompt(label='production')                 |
| Prompt linking to trace   | Manual via metadata                     | Direct:<br>update_current_observation(prompt=) |
| Dataset creation          | create_dataset() +<br>create_examples() | create_dataset() +<br>create_dataset_item()    |
| Running evaluation        | evaluate(target, data, evaluators)      | for item: item.run() + create_score()          |
| Comparative eval          | Built into evaluate() results           | Compare experiment runs in UI                  |
| User feedback             | create_feedback(run_id, key,<br>score)  | create_score(trace_id, name, value)            |
| Session grouping          | session_id in metadata                  | session_id as first-class field on trace       |
| Cost per request          | Token metadata + manual calc            | Automatic from model + token counts            |

# 25. Production Architecture Patterns

## Pattern 1 — LangSmith-First (LangChain Heavy Teams)



## Pattern 2 — Langfuse-First (Privacy / Self-Hosted)



## Pattern 3 — Dual Instrumentation

Some teams use both tools simultaneously during a transition or evaluation period. LangSmith for developer workflow and daily debugging, Langfuse for the governance dashboard and compliance reporting. Both can be active at the same time — pass both a LangSmith tracer and a Langfuse CallbackHandler in the callbacks list.

## The LLMOps Observability Stack

| Layer           | LangSmith             | Langfuse                   |
|-----------------|-----------------------|----------------------------|
| Request tracing | Automatic via env var | CallbackHandler / @observe |

| Layer             | LangSmith                           | Langfuse                             |
|-------------------|-------------------------------------|--------------------------------------|
| Prompt versioning | Hub with commit hashes              | Versioned prompts with labels        |
| Golden datasets   | create_dataset + create_examples    | create_dataset + create_dataset_item |
| Evaluation        | evaluate() function with evaluators | item.run() loop with create_score    |
| Human feedback    | create_feedback()                   | create_score() with any source       |
| Cost monitoring   | Via token metadata + dashboard      | Automatic from model + token counts  |
| Self-hosting      | Not available                       | Docker Compose in 10 minutes         |
| CI/CD integration | evaluate() in GitHub Actions        | run() loop in GitHub Actions         |

## 26. Discussion Questions & Assessment

### Conceptual Questions

12. A support bot worked well for 3 months, then suddenly started giving poor refund advice. Walk through exactly how you would use LangSmith to investigate. What would you check first, second, and third?
13. Your team is debating whether to add few-shot examples to the system prompt or keep it short. How would you use golden datasets and evaluation to make this decision with data rather than opinion?
14. A colleague says: 'We don't need prompt versioning — we just keep our prompts in a Git repository.' What does Git not give you that LangSmith Hub does?
15. Your startup is building a healthcare application that processes patient data. The CTO insists all data must stay on-premises. How does your choice of observability tool change?
16. You run an evaluation and get these results: Prompt v2 average score = 0.88 vs Prompt v1 = 0.84. But looking at individual categories, the 'edge cases' category dropped from 0.76 to 0.61. Do you ship Prompt v2? Justify your answer.
17. Describe the difference between a Score attached to a Trace and a Score attached to a Generation in Langfuse. When would you use each?
18. You have a RAG pipeline where users sometimes report that the answer contradicts information in the knowledge base. Design an evaluator that would detect this automatically. What inputs does it need? What does it output?

### Architecture Questions

19. Design the tracing setup for a multi-agent pipeline where Agent A researches a topic, Agent B critiques the research, and Agent C writes a final summary. How would you structure the traces so the full workflow is visible in one trace?
20. You want to run automated evaluation every night against your golden dataset. Sketch the GitHub Actions workflow. Which LangSmith or Langfuse calls does it make? What happens if the average score drops below your threshold?

### Quick Check — Key Concepts

| Question                                                       | Key Answer                                                                  |
|----------------------------------------------------------------|-----------------------------------------------------------------------------|
| What environment variable enables auto-tracing in LangSmith?   | LANGCHAIN_TRACING_V2=true                                                   |
| What is the difference between a Trace and a Run in LangSmith? | A Trace is the root Run. All other Runs are children of it.                 |
| What does @traceable do for non-LangChain code?                | Creates a Run for every call to that function, inserted into the trace tree |
| What is a golden dataset?                                      | Curated (input, expected_output) pairs that define correct behaviour        |

| Question                                                                       | Key Answer                                                                     |
|--------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| What signature must every evaluator function have?                             | (run, example) → {key, score, comment}                                         |
| In Langfuse, what does as_type='generation' do?                                | Tags the span as an LLM call, enabling token counts and cost display           |
| How does Langfuse link a trace to a prompt version?                            | update_current_observation(prompt=prompt_object) inside @observe               |
| What is the key advantage of Langfuse over LangSmith for regulated industries? | Fully self-hostable — all data stays on your infrastructure                    |
| What does item.run() do in Langfuse?                                           | Opens an experiment run context, linking that trace to the dataset item        |
| LangSmith prompt pull by version?<br>Langfuse prompt pull by label?            | LangSmith: pull_prompt('name:hash') — Langfuse: get_prompt(label='production') |

## End of Teaching Guide

*LangSmith & Langfuse — Day 14 LLMOps Operations | Instructor Use Only*